



ESWACHH INTEGRATED SOLUTIONS PRIVATE LIMITED

Eswachh Platform Documentation

How the service works, and how the software behind it is built. Two references in one document.

PART ONE

Operations Handbook

Managers, support staff and franchise owners

PART TWO

Rebuild Brief

Developers, and the team rebuilding on Vue and Laravel

Operations Handbook

How the platform works, who can see what, and what happens automatically.

Written for the people who run the service day to day.

In this part

- | | |
|-------------------------------|----------------------------|
| 1. What the platform does | 6. Complaints |
| 2. Sectors and franchises | 7. Cleaner attendance |
| 3. Who sees what | 8. What runs on its own |
| 4. The subscription lifecycle | 9. Reading the dashboard |
| 5. Payments and revenue | 10. Questions that come up |
-

01

What the platform does

Eswachh sells doorstep car cleaning as a subscription. A customer signs up for a package against a specific car, pays online through Razorpay, and a cleaner services that car on a schedule until the subscription runs out or is renewed.

Everything in the system hangs off one record: the **order**. An order is not a one-off purchase — it *is* the subscription. It holds the car, the customer, the package, the cleaner assigned to it, the start and renewal dates, and its current status. When people say “62 subscriptions”, they mean 62 order records with an active status.

02

Sectors and franchises

The service area is divided geographically. Each level narrows the one above it:

State → City → Area → Sector → Society

Sector is the level that matters commercially. A franchise owner is granted one or more sectors, and that grant decides everything they can see. Six sectors are live today: Chi 5, Chi 4, Chi 3, Phi 4, Phi 3 and Phi 2.

A franchise can hold several sectors; a cleaner belongs to exactly one. That asymmetry is deliberate. A franchise is a commercial territory that may span sectors, so it is recorded as a mapping table. A cleaner physically works one patch, so their sector sits on their own profile alongside their address.

Every order carries the sector it is serviced in. If a customer moves to a different sector, their orders move with them — the new franchise services the car from that point. Payments already taken do *not* move: revenue stays with the franchise that earned it.

03

Who sees what

ROLE	USES THE SYSTEM TO	SECTOR LIMITED
Super admin	Everything: masters, users, orders, payments, reports	No – sees all
Franchise owner	Run their own territory: orders, customers, cleaners, complaints, attendance, revenue	Yes – own sectors
Cleaner	See today's round, record attendance, close complaints assigned to them	Yes – own work only
Customer	See their own cars, renew, raise complaints	Yes – own records
Supervisor	Legacy role, one account, currently cannot reach the admin area	—

What a franchise owner can and cannot do

They can See and filter their sectors' orders, revenue, customers, cleaners, complaints and attendance. Assign cleaners, including in bulk. Put subscriptions on hold or make them active. Add customers and cleaners into their own sectors.	They cannot See any other sector's data, by any route. Reach the Manage Masters menu. Create administrators or other franchise owners. Correct a payment status by hand. Open a record outside their sectors even with a direct link.
---	---

The restriction is enforced on the server, not in the browser. Choosing a sector from a dropdown does not grant access to it — a request for a sector the person does not own is refused outright. Asking for a record in another sector returns “not found” rather than “forbidden”, so a franchise owner cannot even confirm that a record exists elsewhere.

04

The subscription lifecycle

STATUS	MEANS	CLEANING HAPPENS
Pending	Started but payment never completed	No

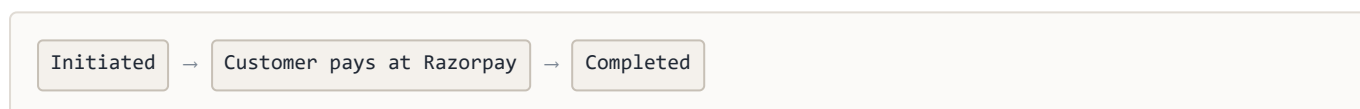
STATUS	MEANS	CLEANING HAPPENS
Active	Paid and running	Yes
Hold	Paused — usually overdue for renewal	No
Deactive	Ended	No

“Expired” is not a fifth status. An expired subscription is one that is still **Active** but whose renewal date has passed. It stays active — and keeps being cleaned — until somebody renews it or the weekly job moves it to Hold. That is why the dashboard shows expired subscriptions separately: they are the ones needing chasing.



05

Payments and revenue



A payment is written the moment the customer is sent to the payment screen, then completed in place when the money arrives. This matters because it makes abandoned payments visible: previously somebody who reached the payment screen and gave up left no trace at all, so there was nothing to follow up.

Revenue means completed payments only. Initiated payments are never counted. Every revenue figure — dashboard tiles, daily and monthly reports — comes from the payment records, not from the amount written on the order.

Correcting a payment by hand

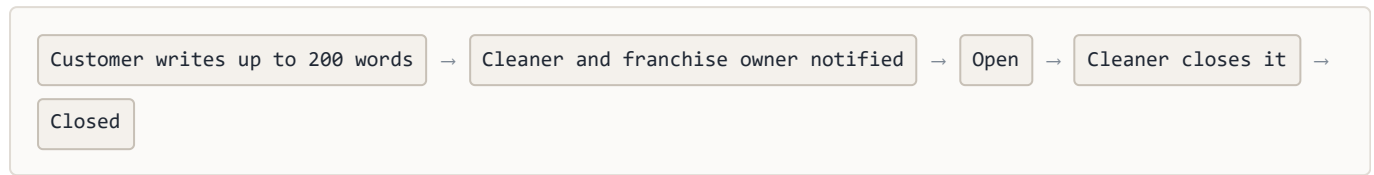
If a payment reaches the bank but the system did not record it as completed, an administrator can set the status manually on the Payments screen. The change records who made it, when, and any note added — a franchise owner cannot do this, deliberately.

Invoices

Each payment has a printable invoice showing the customer, the car, the service period, the package, the amount and the gateway reference. The customer who paid can always open their own invoice; everybody else needs payment access and is held to their sectors.

06

Complaints



The customer picks one of their cars and describes the problem. Raising it sends a WhatsApp message to the cleaner assigned to that car *and* to every franchise owner responsible for that sector.

The cleaner sees only complaints assigned to them. They close each one by choosing **Talked with customer** or **Not talked**, optionally with a note. Either choice closes the complaint — the distinction is a record of whether contact was actually made. A complaint cannot be closed twice, and the first outcome stands.

The cleaner recorded on a complaint is the one who was on that car *at the time*. Reassigning the car later does not move an old complaint to somebody else.

07

Cleaner attendance

A cleaner opens the Attendance page and reports how many of their cars they serviced that day, chosen from a list running from zero up to the number of active cars actually on their round.

CARS SERVICED	RECORDED AS
One or more	Present
None	Absent

Only one record exists per cleaner per day. Submitting again corrects that day rather than adding a second entry. The number of cars on the round is stored with each record, so a figure from six months ago still reads correctly even though the round has changed size since.

08

What runs on its own

JOB	WHEN	WHAT IT DOES
Reminders	Daily, 09:00	WhatsApp to four groups: due to renew within a week, past the renewal date, sitting on hold, and running low on cloths

JOB	WHEN	WHAT IT DOES
Auto hold	Mondays, 09:30	Moves any active subscription more than 7 days past renewal to Hold, and tells the customer
Backup cleanup	Daily, 01:00	Housekeeping on stored backups

The seven-day gap between the first reminder and the automatic hold is intentional: a customer is warned every day for a week before their cleaning stops.

WhatsApp messages are only sent from the live server. On a developer's machine or a test copy they are written to a log file instead. This exists so that a copy of live data — which contains real customer phone numbers — cannot accidentally message hundreds of real people.

09

Reading the dashboard

The sector selector at the top drives every tile. Administrators can pick *All Sectors* or any single one; a franchise owner sees *All My Sectors* and their own.

TILE	COUNTS
Total Cleaner	Cleaners working in the selected sectors
Total Subscriptions	Orders with status Active — includes ones past their renewal date
New Subscriptions	Active, still in date, with no cleaner assigned yet
Expired Subscriptions	Active, but the renewal date has passed
Hold Subscriptions	Orders on Hold
Total Revenue	All completed payments, all time
Revenue This Month	Completed payments in the current calendar month

Because expired subscriptions are still active, *Total Subscriptions* includes them. Expired is a subset, not a separate group.

10

Questions that come up

Can a franchise owner see another franchise's revenue?

No. Every figure is filtered by their sectors before it is calculated, on the server. There is no combination of filters or links that reveals another sector's numbers.

What happens when a customer moves house to another sector?

Their orders move to the new sector, so the new franchise services the car. Past payments stay attributed to the old franchise.

Why can a cleaner only be in one sector?

A cleaner physically services a round in one area. Their sector is stored on their own profile, and it is what decides which franchise owner sees them and can assign them work.

Who can put a subscription on hold?

An administrator, or a franchise owner within their own sectors. The weekly job also does it automatically once a subscription is a week overdue.

A customer says they paid but their subscription did not renew.

Look them up on the Payments screen. If the payment shows as *Initiated*, the money did not reach us and they should try again. If it shows *Completed* but the subscription did not update, the payment is recorded and an administrator can correct the order — the money is never lost from the records.

Rebuild Brief

The domain model, the business rules that must survive, and the traps in the current codebase. Written for the team rebuilding this on Vue and Laravel.

In this part

- | | |
|----------------------------|------------------------------|
| 1. What exists today | 6. Notifications |
| 2. Domain model | 7. Scheduled jobs |
| 3. Rules that must survive | 8. Traps in the current code |
| 4. Authorization | 9. Suggested API surface |
| 5. Payments | 10. Carrying the data over |

01

What exists today

Laravel 10.38 on PHP 8.2, built on the `nasirkhan/laravel-starter` template, with domain code split across `nwidart/laravel-modules` packages in `Modules/`. Roles and permissions come from `spatie/laravel-permission`. Lists are server-side Yajra DataTables. UI is Blade with CoreUI, `spatie/laravel-html` builders and select2. Payments are Razorpay; messaging is MSG91 WhatsApp.

Around 18 modules exist but only a handful carry logic. The pattern is consistent: a controller extends `BackendBaseController`, sets five properties and inherits generic CRUD. Newer screens — Payments, Complaints, Attendance — sit in `app/Http/Controllers/Backend` instead, because they are cross-cutting rather than a single entity's CRUD.

The migrations do not describe the production schema. Columns were added by hand over two years, so `orders` in the database has roughly thirty columns while its migration creates six. A `database/schema/mysql-schema.sql` dump now captures the real structure — treat that dump, not the migration files, as the source of truth.

02

Domain model

The order is the subscription

There is no separate subscriptions table. An `orders` row is the subscription: customer, car, package, assigned cleaner, `start_date`, `renew_date`, money paid,

status. Renewal mutates that same row and pushes `renew_date` forward. Historic states are snapshotted as JSON into `order_history`.

TABLE	HOLDS	NOTES FOR THE REBUILD
<code>orders</code>	Subscriptions	Soft deletes. <code>status</code> 1/2/3/4. <code>sector_id</code> added 2026-08.
<code>users</code>	All humans	Customers, cleaners, franchise owners, admins — one table, separated by role.
<code>userprofiles</code>	Address and location	One row per user. Carries <code>sector_id</code> for customers <i>and</i> cleaners.
<code>sector_user</code>	Franchise → sectors	Many-to-many. The only place a franchise's territory is defined.
<code>payment_history</code>	Money	Source of truth for revenue. Has <code>sector_id</code> , <code>payment_id</code> , <code>razorpay_order_id</code> .
<code>complaints</code>	Customer complaints	Stamped with sector and the cleaner at time of complaint.
<code>cleaner_attendances</code>	Daily attendance	Unique on <code>(user_id, date)</code> .
<code>cleaner_activities</code>	Per-car daily cleaning	Distinct from attendance — this is “was this car washed”.
<code>states / cities / areas / sectors / societies</code>	Location hierarchy	Plain integer columns, no FK constraints.

```
states → cities (city.state_id) → areas (area.city_id)
      → sectors (sector.area_id) → societies (society.sector_id)
```

Sector is the tenancy boundary. Everything else in the chain is addressing detail. Model sector as a first-class scope, not a lookup value.

A profile that carries only a sector breaks the address UI. The legacy profile form rebuilds its Sector dropdown by cascading state → city → area → sector, so a record with `sector_id` but no parents shows an empty box. Always store the whole chain, and derive the parents when you only have the leaf.

Rules that must survive

1. **Expired is derived, not stored.** `status = 2` with `renew_date < today`. There is no expired status.

2. **Orders follow the customer; payments do not.** A profile sector change moves their orders. Recorded payments keep the sector they were taken in.
3. **A franchise holds many sectors; a cleaner holds exactly one.**
4. **Revenue is completed payments only.** The stored status string is `captured` — renaming it silently zeroes every revenue figure.
5. **The cleaner on a complaint is frozen at creation.**
6. **Attendance is one row per cleaner per day.** `total_cars` is denormalised so historic rows stay meaningful.
7. **Seven days of grace before auto-hold.**

04

Authorization

Layer one — permissions

Spatie roles and permissions named `{ability}_{resource}`. A trait overrides `callAction()` and authorizes automatically by mapping the controller method to an ability. Super admin bypasses everything via `Gate::before`.

Layer two — sector scoping

Permissions say *what* you can do; sectors say *whose records*. All of it funnels through

`App\Services\SectorService`:

```
allowedSectorIds()      null = unrestricted, array = limited, [] = sees nothing
selectedSectorIds($id)  validates a requested sector, aborts 403 if not yours
scopeByUserSector()     limits a users query via userprofiles.sector_id
canAccessSector($id)    single-record check
locationChainFor($id)   sector -> area, city, state
```

Fail closed, and do not disclose existence. A franchise owner with no sectors sees nothing, not everything. Records outside your sectors return **404, not 403**. The requested sector from the browser is always validated server-side; the dropdown is a convenience, never the control.

The single biggest trap in this codebase. Role checks are written as `$user->roles()->pluck('name')[0]` — first role only — in a branch shaped `if customer ... elseif cleaner ... else see everything`. Any new role silently lands in the admin branch. It appears in at least nine places. Use explicit capability checks and never an `else` that means unrestricted.

05

Payments

Razorpay, four entry points: new subscription, renewal while logged in, renewal without login via a signed token, and cloth top-up. They converge on the same shape, centralised in `App\Services\RazorpayService`.

```
1 create Razorpay order          server-side price calculation
2 record payment as 'initiated'  so abandonment is visible
3 customer pays at Razorpay
4 callback arrives
5 verify HMAC signature
6 reject if payment id already processed (idempotency)
7 record/complete the payment record (money first)
8 update the subscription        (inside a transaction)
9 notify                         (outside the transaction)
```

Order of steps 7 and 8 is deliberate. The money has already left the customer, so it is recorded before the subscription is touched; if the update then fails, the payment is on file and reconcilable rather than lost.

Hard-won details

- **Razorpay notes are strings.** A select with nothing chosen posts the literal `"null"`, which `??` does not catch. Writing that to an integer column throws.
- **Verify the signature.** The original code never did.
- **Idempotency needs the gateway payment id on the payment record.** Without it a resubmitted callback renews twice and banks the money twice.
- **Never echo an exception to the customer.** The original `catch` did `return $e->getMessage()`, printing SQL to the payment page and skipping the payment insert.
- `php artisan payments:reconcile --days=30` finds captured payments never recorded.

06

Notifications

WhatsApp via MSG91 templates: `SMSService::sendWhatsAppMsg($mobile, $template, $variables)`. Templates are referenced by name and must exist in the MSG91 account, including the newer `complaint_notification_cleaner` and `complaint_notification_franchise`.

Delivery is environment-gated. With nothing configured, only production sends; everywhere else the message goes to the log, variables included, which is how OTP signup works on a developer machine. Keep this — a copy of live data carries real phone numbers.

07

Scheduled jobs

COMMAND	SCHEDULE	PURPOSE
<code>renewal:send-notifications</code>	Daily 09:00	Due soon, expired, on hold, low cloths
<code>orders:auto-hold</code>	Mon 09:30	Hold anything 7+ days past renewal
<code>payments:reconcile</code>	Manual	Find captured payments never recorded
<code>backup:clean</code>	Daily 01:00	Backup housekeeping

Both business jobs take `--dry-run`. Keep that: it is the only safe way to inspect them against a copy of production.

08

Traps in the current code

ISSUE	SEVERITY	STATE
Role and permission accessors shadowed Spatie's relations and cached every assignment with <code>rememberForever</code> . New users had no roles, and role changes stayed invisible, until the cache was cleared by hand — producing 403s on login.	HIGH	Fixed
<code>StartSession</code> registered twice. Flashed data is lost, so failed form submissions show no error.	HIGH	Open
Tests used <code>RefreshDatabase</code> with no test-database override, so the suite ran <code>migrate:fresh</code> against whatever <code>.env</code> pointed at.	HIGH	Fixed
Gateway keys read via <code>env()</code> at runtime; the first <code>config:cache</code> makes every payment silently fail.	HIGH	Fixed
A customer's temporary password was written to the log in plaintext.	HIGH	Fixed
Payment callbacks had no signature verification and no idempotency.	HIGH	Fixed
<code>payment_date_time</code> carried <code>ON UPDATE CURRENT_TIMESTAMP</code> , so any write rewrote the payment date.	MEDIUM	Fixed
DataTables endpoints resolved relations per row: ~200 extra queries per page.	MEDIUM	Fixed
No indexes beyond primary keys on the busiest tables.	MEDIUM	Fixed
The reminder job messaged soft-deleted orders.	MEDIUM	Fixed
OPcache disabled; app served by single-threaded <code>artisan serve</code> .	MEDIUM	OPcache enabled

ISSUE	SEVERITY	STATE
<code>cleaners/create.blade.php</code> is dead scaffolding that posts to <code>roles.store</code> .	LOW	Open
Two screens edit one user, splitting name and address from account and roles.	LOW	Open
Three orphaned <code>userprofiles</code> rows point at a sector that does not exist.	LOW	Open

The two user screens

Account fields live on one screen and identity plus address on another. Email appears on both. The rebuild should present **one user record with sections** — Account, Profile & Address, Abilities — saved in one submit. Note the profile screen doubles as self-service: customers and cleaners edit their own details through it, so the new design needs one form with fields shown or hidden by permission, not two routes.

09

Suggested API surface

```

GET    /api/me                user, roles, permissions, allowed sectors
GET    /api/sectors          only sectors the caller may use

GET    /api/orders?sector_id=&status=&renew_from=&renew_to=
PATCH /api/orders/{id}      status, assigned cleaner
POST   /api/orders/bulk-assign { ids[], cleaner_id }

GET    /api/dashboard?sector_id=    all tiles in one response
GET    /api/payments?sector_id=&status=&from=&to=
PATCH /api/payments/{id}/status  admin only, audited
GET    /api/payments/reports?group=daily|monthly
GET    /api/payments/{id}/invoice

GET    /api/complaints?status=&sector_id=
POST   /api/complaints        customer, max 200 words
PATCH /api/complaints/{id}/resolve { resolution, note }

GET    /api/attendance?date=&sector_id=
POST   /api/attendance        cleaner, { cars_served }
```

- Return `allowed_sectors` from `/api/me` so the front end can render the selector — but keep validating the sector on every request.
- Give the dashboard one endpoint. The current page issues six counts plus two sums.
- Keep the Razorpay callback as a server-rendered POST endpoint, not an SPA route: it is a browser redirect from the gateway and the signature check must happen server-side.
- Model status as an enum, but keep `captured` as the stored payment value so historic rows and revenue queries stay valid.

Carrying the data over

Roughly 1,970 orders, 500 users, 500 payments — small enough that a scripted migration in one pass is realistic.

- **Start from the schema dump**, not the migration files.
- **Backfill sector on orders from** `userprofiles.sector_id`, and on payments from their order. Roughly 7 orders have no derivable sector: five are abandoned checkouts with no `user_id`, two are inactive customers who never had a sector recorded. Leave them unassigned and admin-only.
- **Split the users table conceptually** into person plus role-specific data, but keep one identity.
- **Preserve** `order_history` JSON snapshots; they are the only audit trail for subscription changes.
- **Reconcile payments before cutover** so the new system does not inherit a gap.
- **Watch the timezone.** `payment_date_time` was written by PHP in app time while `created_at` defaulted to MySQL server time — the two differ by about six hours on the same row.